

Firmware Design — The Big "What the Hell is Going On" Guide

An in-depth walkthrough of the course syllabus

Course companion notes

May 2026

Contents

1	Classic vs. Embedded Systems	5
1.1	Hardware differences	5
1.2	Software differences	5
2	Microprocessor Interrupts Management	6
2.1	Why interrupts at all?	6
2.2	The mechanism, step by step	6
2.3	Important concepts	6
3	Executable Code Generation — The GCC Model	7
3.1	The four stages	7
3.2	Stopping at each stage	8
3.3	objdump: read the binaries you produce	8
4	Cross-Compilation Environments for Embedded Systems	8
4.1	Why cross-compile?	9
4.2	The toolchain triplet	9
4.3	What changes vs. native gcc?	9
5	Sections in Object Files and Executables	9
5.1	The canonical sections	10
5.2	Why this matters in embedded	10
5.3	Custom sections via attributes	10
6	Memory Allocation of Static vs. Local Variables	10
6.1	Static variables	10
6.2	Local (automatic) variables	11

6.2.1	Without optimisations (<code>-O0</code>)	11
6.2.2	With optimisations (<code>-O2</code> , <code>-Os</code> , <code>-O3</code>)	11
6.3	RISC vs. CISC variations	11
7	Debug Hardware Unit in Microcontrollers	12
7.1	The pieces (ARM CoreSight family)	12
7.2	Hardware vs. software breakpoints	12
7.3	What you can do thanks to all this	12
8	Dynamic Memory Allocation in the C Standard Library	13
8.1	What <code>malloc</code> actually does	13
8.2	Embedded reality: <code>newlib</code> and <code>_sbrk</code>	13
8.3	Why we don't trust <code>malloc</code> in embedded	13
9	C Constructs to Avoid in Embedded	14
9.1	Always use braces, even for single-statement blocks	14
9.2	No unlimited recursion	14
9.3	No <code>malloc/free</code>	14
9.4	Prefer indexes to pointers for array traversal	15
9.5	Other classics worth knowing	15
10	Dedicated Memory Areas for I/O Buffers and DMA Access	15
10.1	Why DMA needs special memory placement	15
10.2	An example layout	16
11	The <code>const</code> and <code>volatile</code> Type Modifiers	16
11.1	<code>const</code> : “the program promises not to write through this”	16
11.2	<code>volatile</code> : “the value may change without the compiler knowing”	16
11.3	<code>const volatile</code> — yes, really	17
11.4	Famous <code>volatile</code> mistakes	17
12	Mandatory Content of <code>.c</code> and <code>.h</code> Files	17
12.1	Header (<code>.h</code>) file content	17
12.2	Source (<code>.c</code>) file content	18
12.3	Function prototyping	18
13	Peripheral Representation in the Memory Map	18
13.1	Three flavours of register	19
13.2	Accessing registers from C: the typedef-struct trick	19

13.3 Bit fields you'll meet	19
14 STM32CubeIDE	20
14.1 What's inside	20
14.2 Typical workflow	20
14.3 FreeRTOS integration	20
15 HAL vs. LL Libraries; Callbacks; Weak Symbols	21
15.1 HAL (Hardware Abstraction Layer)	21
15.2 LL (Low-Level)	21
15.3 Interrupt handling through HAL/LL	21
15.4 Callbacks: the concept	22
15.5 Weak symbols	22
16 Sensor Interfacing: NTC Thermistor Example	22
16.1 NTC: what it is	22
16.2 Resistance-to-voltage: the voltage divider	23
16.3 Design choices	23
17 Fixed-Point vs. Floating-Point	23
17.1 Floating-point: IEEE 754	24
17.2 Fixed-point: integers with an implied scale	24
17.3 When to use which	24
18 FreeRTOS Best Practices: Tasks, Queues, Events	24
18.1 Tasks	25
18.2 Queues	25
18.3 Events and other primitives	25
19 Timer Peripheral: Prescaler and Period	26
19.1 The two knobs	26
19.2 Worked example	26
19.3 What you actually do with timers	26
20 Power Electronics from Scratch	27
21 Load Switches: Low-Side and High-Side	27
21.1 Low-side switch	28
21.2 High-side switch	28
21.3 NPN BJT as low-side switch: detailed walk-through	28

22 Integrated High-Side Drivers	29
22.1 Why turn-on/turn-off speed matters	29
23 Switch Power Dissipation; NPN Base Resistor Sizing	29
23.1 Where the power goes (MOSFET)	29
23.2 Where the power goes (BJT)	30
23.3 Sizing an NPN base resistor	30
24 Full-Bridges and Shoot-Through	30
24.1 Topology	31
24.2 Shoot-through: the cardinal sin	31
24.3 Protection: dead-time	31
24.3.1 Timer-based dead-time	31
24.3.2 Driver-based dead-time	31
25 Digital-Interface Sensors: The Accelerometer Example	32
25.1 Why digital is nicer	32
25.2 MEMS accelerometer, briefly	32
25.3 Example: hard disk drives and free-fall detection	32
26 I²C and SPI Protocols	32
26.1 I ² C (Inter-Integrated Circuit)	32
26.2 SPI (Serial Peripheral Interface)	33
26.3 Comparison cheat sheet	34
26.4 In an STM32 (peripherals view)	34
Closing Thoughts	34

1 Classic vs. Embedded Systems

A “classic” data processing system is what you are reading these notes on: a PC or a server. An **embedded system** is a computer that you usually don’t see as a computer: the thing inside your washing machine, car ECU, drone flight controller, smart thermostat, or pacemaker. Both run code, but they live in very different worlds.

1.1 Hardware differences

Aspect	Classic (PC/Server)	Embedded (MCU)
CPU	GHz, multi-core, out-of-order, deep pipeline, branch predictor, big caches	MHz to few hundred MHz, often single-core (Cortex-M, AVR), shallow pipeline, little or no cache
Memory	GB of DRAM, virtual memory via MMU	KB to a few MB of <i>on-chip</i> SRAM; no MMU (usually an MPU at most)
Storage	TB of SSD/HDD with a file system	KB–MB of internal Flash; sometimes external SPI Flash; no FS unless you bolt one on
Peripherals	PCIe, USB, NIC; abstracted by OS drivers	GPIO, UART, SPI, I ² C, ADC, DAC, timers, CAN; you talk to them register-by-register
Power	10s–100s of W, fans, heatsinks	mA or μ A; often battery powered, sometimes energy harvesting
Cost	100s–1000s of \$, depreciates over years	cents to a few \$, sold by the million

1.2 Software differences

- **Operating system.** A PC has Linux/Windows/macOS with a kernel, drivers, multitasking, virtual memory, security boundaries, paging, etc. An MCU often runs *bare-metal* (your code *is* everything), or at most a small RTOS (FreeRTOS, Zephyr, ThreadX) that gives you tasks, queues and mutexes but no MMU and no protected processes.
- **Real-time constraints.** A PC’s job is throughput (“finish quickly on average”). An embedded system frequently has *deadlines* (“never miss a sample window or the motor melts”). A 1 ms hiccup on Linux is invisible; on a brushless-motor controller it can be a fire.
- **Determinism.** You avoid features that introduce unpredictability: dynamic memory allocation, garbage collection, unbounded loops/recursion, virtual memory faults, etc.
- **Resource consciousness.** Every byte of Flash and every byte of RAM matters. You profile not just runtime but *stack depth* and *ROM usage*.
- **Update model.** On a PC you “apt upgrade”. On an embedded device you must design a firmware update mechanism (bootloader, dual-bank flash, signature checking) or accept that the firmware is frozen forever.
- **Failure model.** On a PC, the OS catches and reports faults. On a bare-metal MCU, a wild pointer either silently corrupts memory or triggers a HardFault that, if not handled, locks the chip — often the only feedback is “the LED stopped blinking”.

Key Idea

An embedded engineer writes code that runs *forever*, on hardware that *costs cents*, against *real-world physics*, with *no operator* to restart the process. That changes every single engineering decision compared to PC software.

2 Microprocessor Interrupts Management

An **interrupt** is a hardware-triggered jump in the CPU's normal execution flow. The CPU is doing something, an event happens (a UART byte arrives, a timer expires, a button is pressed), and the hardware forces the CPU to stop what it was doing, save its state, jump to a piece of code called the **Interrupt Service Routine (ISR)**, run it, then resume the original work as if nothing happened.

2.1 Why interrupts at all?

The alternative is **polling**: a loop that constantly checks each peripheral, “do you have data? do you have data? do you have data?”. Polling wastes CPU, increases latency to the worst-case polling period, and makes power management impossible (CPU can never sleep). Interrupts let the CPU sleep or do useful work and react only when needed.

2.2 The mechanism, step by step

1. Peripheral asserts an IRQ line (e.g. “USART2_RX has a byte”).
2. The Interrupt Controller (**NVIC** on ARM Cortex-M) prioritises pending interrupts and, if priority allows, signals the core.
3. The core finishes the current instruction, then automatically pushes a context (on Cortex-M: xPSR, PC, LR, R12, R3, R2, R1, R0) onto the current stack. This is the *hardware-stacked frame*.
4. The core loads the PC from the **Interrupt Vector Table** entry corresponding to that IRQ number. The vector table is just an array of function pointers at a fixed memory address (start of Flash on Cortex-M).
5. The ISR runs. It must clear the source of the interrupt (e.g. read the data register, write the “flag clear” bit), otherwise the interrupt re-fires forever.
6. ISR returns by executing a special return value (**EXC_RETURN** on Cortex-M). Hardware unstacks the frame and resumes the interrupted code.

2.3 Important concepts

Maskable vs. Non-Maskable (NMI).

Maskable IRQs can be globally disabled (**cpsid i** on Cortex-M, **cli** on AVR). NMI cannot be masked — reserved for catastrophic things (clock failure, watchdog pre-warn).

Priority & preemption.

Each interrupt has a priority. A higher- priority interrupt can preempt a lower-priority ISR currently running. Same-priority interrupts do not preempt each other; they queue.

Re-entrancy.

If you call a function from both an ISR and main code, it must be re-entrant: no static state, or properly protected with critical sections. `strtok`, `rand`, `malloc` are not re-entrant.

Latency.

Interrupt latency = time from event to first ISR instruction. Cortex-M3/M4: 12 cycles deterministic, including the hardware stacking. This determinism is the whole point of Cortex-M for real-time.

Tail-chaining & late arrival.

Cortex-M optimisations: if a new IRQ is pending when an ISR finishes, the unstack/re-stack is skipped (tail-chaining). If a higher-priority IRQ arrives during the stacking of a lower one, the higher one runs first (late arrival).

Watch Out

The #1 ISR mistake: doing too much work inside it. ISRs should be *short*. Read the byte, push it to a queue, signal a task, return. Long ISRs increase worst-case latency for all other interrupts and make the system look fine in testing and explode in the field.

```

1 volatile uint8_t rx_buf[64];
2 volatile uint8_t rx_head = 0;
3
4 void USART2_IRQHandler(void)
5 {
6     if (USART2->SR & USART_SR_RXNE) {
7         rx_buf[rx_head++ & 0x3F] = USART2->DR; /* reading DR clears RXNE */
8     }
9 }

```

Listing 1: Typical ISR pattern (STM32 HAL)

Notice the `volatile`: the ISR and main code share `rx_head` and `rx_buf`, so the compiler must not cache these in registers across uses.

3 Executable Code Generation — The GCC Model

A C program does not magically become an executable. GCC orchestrates *four* distinct stages, each of which can be stopped and inspected.

3.1 The four stages

`.c` $\xrightarrow{\text{preprocess}}$ `.i` $\xrightarrow{\text{compile}}$ `.s` $\xrightarrow{\text{assemble}}$ `.o` $\xrightarrow{\text{link}}$ `a.out` / `.elf`

1. **Preprocessing** (`cpp`). Expands `#include`, `#define`, `#if`, removes comments. Output is still C, but with all headers inlined and macros substituted. No compilation yet.
2. **Compilation** (`cc1`). Parses the C, does optimisations, emits *assembly* for the target architecture. This is where things like inlining, dead-code elimination, and register allocation happen.
3. **Assembly** (`as`). Turns assembly mnemonics into machine code, producing a relocatable **object file** (`.o`). Object files contain code and data, plus a *symbol table* and *relocation entries* (placeholders to be patched at link time).

4. **Linking** (`ld`). Combines all the `.o` files and libraries, resolves cross-references, lays out the final memory map according to a *linker script*, and emits the executable (ELF on Linux / embedded, PE on Windows, Mach-O on macOS).

3.2 Stopping at each stage

Listing 2: Useful gcc invocations

```
# Preprocess only -- look at the expanded source
gcc -E hello.c -o hello.i

# Compile to assembly -- look at what gcc emitted
gcc -S hello.c -o hello.s
gcc -S -O2 hello.c -o hello_opt.s # with optimisation

# Compile + assemble, but don't link -- get the .o
gcc -c hello.c -o hello.o

# Full pipeline
gcc hello.c -o hello

# Useful diagnostic flags
gcc -Wall -Wextra -Wpedantic -O2 -g hello.c -o hello
gcc -v hello.c # verbose: show every sub-tool invocation
gcc -save-temps hello.c # keep .i and .s alongside the output
```

3.3 objdump: read the binaries you produce

`objdump` is to object files what `less` is to text files. You should reach for it constantly when “something is weird”.

Listing 3: objdump cheat sheet

```
objdump -h hello.o # list sections and their sizes
objdump -d hello.o # disassemble code sections
objdump -D hello.o # disassemble EVERYTHING (including data)
objdump -t hello.o # symbol table
objdump -r hello.o # relocation entries (what the linker still has to patch)
objdump -s hello.o # hex+ASCII dump of all sections (good for .rodata)
objdump -x hello.o # everything: headers, symbols, relocations
objdump -S hello.o # interleave source with disasm (needs -g at compile)

# For embedded: pick the right toolchain prefix
arm-none-eabi-objdump -d firmware.elf
```

Key Idea

If you can read what `objdump -d` produces, you can debug almost any “the C code looks right but the chip does the wrong thing” problem. Optimisers are aggressive — the only ground truth is the disassembly.

4 Cross-Compilation Environments for Embedded Systems

A native compiler runs on *the same* architecture it produces code for: `gcc` on x86 producing x86 binaries. A **cross-compiler** runs on one architecture (the *host*, usually x86_64) and produces code for another (the *target*, usually ARM, RISC-V, AVR, etc.).

4.1 Why cross-compile?

Because the target MCU is almost certainly too small to host a C compiler: your STM32F4 has 192 KB of RAM; GCC needs hundreds of MB. So you build on your laptop, then transfer the resulting `.elf/.bin` to the chip via a debugger probe (ST-Link, J-Link) or a bootloader (DFU, UART).

4.2 The toolchain triplet

A toolchain is named by a *triplet* of the form `arch-vendor-os-abi`. For embedded ARM you'll see:

- `arm-none-eabi-` — ARM CPU, no OS (**none**), EABI calling convention. Used for *bare-metal* and RTOS firmware on Cortex-M / Cortex-R. This is the ST/CubeIDE default.
- `arm-linux-gnueabi-` — ARM CPU, Linux user-space, glibc, hard-float ABI. Used when targeting embedded Linux boards (Raspberry Pi, BeagleBone).
- `riscv64-unknown-elf-` — RISC-V 64, bare-metal.

Each tool is prefixed: `arm-none-eabi-gcc, -as, -ld, -objdump, -objcopy, -gdb, -size`.

4.3 What changes vs. native gcc?

- You must tell it the exact CPU and FPU: `-mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard`.
- There is no `libc` that talks to an OS; you use **newlib** or **newlib-nano**, which expects you to provide *syscalls* (`_write, _sbrk, _read, ...`).
- You must supply a *linker script* (`.ld`) describing Flash and RAM regions.
- You must supply *startup code* (`startup_stm32xxx.s`) that sets up the stack, zeroes `.bss`, copies `.data` from Flash to RAM, then calls `main()`.
- The output `.elf` usually has to be converted to raw binary or Intel HEX for the flasher: `arm-none-eabi-objcopy -O binary fw.elf fw.bin`.

5 Sections in Object Files and Executables

An ELF binary is divided into **sections**, each holding one “kind” of content. The linker decides where each section lives in memory.

5.1 The canonical sections

Section	What's in it	Lives in (MCU)
<code>.text</code>	Executable code (machine instructions)	Flash
<code>.rodata</code>	Read-only data: string literals, <code>const</code> globals, lookup tables	Flash
<code>.data</code>	Initialised, writable globals/statics (e.g. <code>int x = 5;</code>)	RAM (loaded from Flash at boot)
<code>.bss</code>	Zero-initialised globals/statics (e.g. <code>int y;</code>)	RAM (zeroed at boot)
<code>.stack</code>	Function call frames, local variables	RAM
<code>.heap</code>	Dynamically allocated blocks (<code>malloc</code>)	RAM
<code>.isr_vector</code>	Interrupt vector table	Beginning of Flash

5.2 Why this matters in embedded

On a PC you almost never think about sections. On an MCU you do, because:

- Flash and RAM are *separate physical memories* at different addresses. The linker must tell each section which one it lives in.
- `.data` is tricky: it needs an initial value at boot, but it must be writable. Solution: a *copy* is stored in Flash, and the startup code copies it to RAM before `main()`.
- Sections let you place specific code/data at specific addresses (e.g. DMA buffer in a particular SRAM bank — see Section 10).

5.3 Custom sections via attributes

```

1  /* Put this buffer in section ".dma_buffer", defined in linker script */
2  __attribute__((section(".dma_buffer"))) uint8_t dma_buf[1024];
3
4  /* Put a function in RAM for speed (must be copied at boot) */
5  __attribute__((section(".ramfunc"))) void fast_isr(void) { ... }
```

Listing 4: Placing variables in custom sections

The linker script then has matching lines like:

```
.dma_buffer (NOLOAD) : { *(.dma_buffer) } > SRAM2
```

6 Memory Allocation of Static vs. Local Variables

This is a topic where “intuition from PC code” lies to you. Let’s be precise.

6.1 Static variables

Static = *static storage duration*. These exist for the whole program lifetime. That includes:

- File-scope globals (`int counter;`).
- File-scope `static` globals (same lifetime, file-local visibility).

- Function-scope **static** locals (`static int n = 0;` inside a function — exists across calls, only initialised once).

Where do they live?

- Initialised to nonzero → `.data` (RAM at run-time, initial value sitting in Flash).
- Initialised to zero or uninitialised → `.bss` (zeroed RAM, no Flash cost for initial values).
- `const` qualified → `.rodata` (Flash on MCU).

6.2 Local (automatic) variables

A function-local non-**static** variable has *automatic* storage: created on function entry, destroyed on return. Conceptually on the **stack**. *In practice, the optimiser is in charge.*

6.2.1 Without optimisations (-O0)

Every local lives on the stack at a fixed offset from the frame pointer. You can put a breakpoint and inspect any variable at any moment. Code is slow and bloated but predictable. This is what you want when stepping through code in a debugger.

6.2.2 With optimisations (-O2, -O3)

- Locals that fit in registers and have short live ranges are kept *entirely in registers* — they never touch RAM.
- Locals whose value is never used after assignment are *eliminated*.
- Constants computed at compile time are propagated into uses (constant folding).
- Dead code (`if (0) { ... }`) is removed.
- Loops may be unrolled or rewritten.

Watch Out

A consequence that bites every beginner: in optimised code, a variable `x` may not exist at the moment you place a breakpoint. The debugger shows “optimized out”. Use `-Og` (debug-friendly optimisation) or `-O0` while learning. Also: an optimiser may remove a delay loop unless variables are `volatile` or you use a real timer.

6.3 RISC vs. CISC variations

RISC (ARM Cortex-M, RISC-V, AVR). Load-store architecture: ALU operates only on registers. To use a memory value you must **LOAD** it, operate, then **STORE**. RISCs have many general-purpose registers (ARM: 13 of them in user mode). ⇒ Compilers keep many locals in registers. Function prologues push the registers they intend to clobber; epilogues pop them.

CISC (x86, x86-64). Memory operands allowed in most instructions: `ADD [ebp-4], eax` adds `eax` directly into the local at `[ebp-4]`. x86-32 has only ~8 GP registers, so locals more often spill to the stack. x86-64 doubled the register count, so modern x86-64 code looks more RISC-like in its allocation patterns.

Practical impact. On a Cortex-M, a leaf function with five `int` locals usually generates *zero stack usage*: everything in R0–R12. On a 32-bit x86 the same function might push four of them to the stack. This is why embedded stack budgets are often surprisingly small.

7 Debug Hardware Unit in Microcontrollers

On a PC, the debugger (gdb) talks to the OS, which uses `ptrace` to control the target process. On an MCU there is no OS, so the debugger must talk to *hardware logic inside the chip* that can halt the core, peek at registers, set breakpoints, and resume.

7.1 The pieces (ARM CoreSight family)

DAP (Debug Access Port).

The bridge between an external probe (ST-Link, J-Link) and the chip's internal debug bus.

JTAG / SWD.

The wire-level protocols the probe uses. JTAG = 4–5 wires (TCK, TMS, TDI, TDO, optional TRST), industry classic. SWD = 2 wires (SWCLK, SWDIO), ARM-specific, saves pins — this is the common one on modern STM32.

FPB (Flash Patch & Breakpoint).

Hardware breakpoint comparators. Limited (typically 4–8). The CPU halts on instruction fetch from a matching address.

DWT (Data Watchpoint & Trace).

Watchpoints (halt when an address is read/written), plus cycle counters, exception trace, PC sampling.

ITM (Instrumentation Trace Macrocell).

Lets the firmware emit `printf`-style messages over a single SWO pin to the debugger. Insanely useful, costs almost nothing.

ETM (Embedded Trace Macrocell).

Streams the full instruction trace off-chip. Expensive in pin count, used for hard-to-reproduce timing bugs.

MPU (Memory Protection Unit).

Not strictly a debug feature, but pairs with debug: catches accesses to regions you've marked off-limits.

7.2 Hardware vs. software breakpoints

- **Hardware breakpoints** use FPB comparators. They work even on code in Flash (which you cannot modify at runtime). Limited number.
- **Software breakpoints** are implemented by overwriting an instruction with `BKPT` (Cortex-M) or `INT3` (x86), and restoring it on hit. Unlimited, but require writable code memory — so on Flash they fall back to hardware breakpoints.

7.3 What you can do thanks to all this

- Halt the CPU and inspect every register and any memory location.

- Single-step instructions or source lines.
- Set a breakpoint that triggers only when `rx_head == 64`.
- Watch a global and halt the moment something writes to it (great for “who is corrupting my buffer?”).
- Reflash the chip directly from the IDE.
- Stream `printf` over SWO without using a UART.

8 Dynamic Memory Allocation in the C Standard Library

`malloc()` and `free()` are part of the C library, not the language. Their implementation is up to whoever shipped your libc.

8.1 What malloc actually does

1. Maintains an internal data structure tracking free chunks in a pool of memory called the **heap**.
2. On request, walks the free list, picks (or splits) a chunk big enough, marks it allocated, and returns a pointer just past the bookkeeping header.
3. `free()` returns the chunk to the free list and (if possible) coalesces it with adjacent free chunks.
4. When the heap is exhausted, `malloc` asks the OS (or, in embedded, an underlying `_sbrk`) for more memory.

8.2 Embedded reality: newlib and _sbrk

Newlib’s `malloc` doesn’t know your chip’s memory map. It calls `_sbrk(size)`, a function *you* (or the framework) provide, that hands out memory from a region typically defined in the linker script as `_end` up to `_estack - stack_reserve`.

```
1 extern char _end;           /* end of .bss, start of heap (from linker) */
2 extern char _estack;        /* top of RAM (initial stack pointer) */
3 static char *heap_ptr = &_end;
4
5 void *_sbrk(int incr)
6 {
7     char *prev = heap_ptr;
8     if (heap_ptr + incr > (&_estack - 0x400)) { /* 1 KiB stack reserve */
9         errno = ENOMEM;
10        return (void *) -1;
11    }
12    heap_ptr += incr;
13    return prev;
14 }
```

Listing 5: Typical `_sbrk` for STM32 with newlib

8.3 Why we don’t trust malloc in embedded

- **Non-determinism.** Allocation time depends on the current free-list state. A real-time loop can’t tolerate this.

- **Fragmentation.** After many allocate/free cycles, the heap is full of small holes. A new allocation may fail even though the total free memory is enough. There is no compaction (you can't move allocated blocks — pointers would dangle).
- **No graceful failure mode.** If `malloc` returns `NULL` on an MCU, what do you do? On a PC you log and bail out. On a motor controller, “log and bail” may mean “the motor keeps spinning while the firmware is dead”.
- **Heap/stack collision.** Heap grows up, stack grows down, they share the same RAM. Worst case: heap silently overwrites stack or vice versa — HardFault, far from the actual cause.
- **Code size.** Even if you “never call malloc”, linking one function that calls `printf("%f", ...)` can drag in malloc and float support and balloon your binary.

Key Idea

In safety-critical embedded code (automotive MISRA, aerospace DO-178C), dynamic allocation is either entirely forbidden, or restricted to the initialisation phase only.

9 C Constructs to Avoid in Embedded

These rules look like nitpicks. They aren't. They are scar tissue from real, expensive failures.

9.1 Always use braces, even for single-statement blocks

```
1  /* BAD */
2  if (flag)
3      do_thing();
4      log_event();           /* always runs! indentation lies */
5
6  /* GOOD */
7  if (flag) {
8      do_thing();
9  }
10 log_event();
```

This is exactly the bug behind Apple's *goto fail* TLS vulnerability (2014): a duplicated `goto fail`; not guarded by braces skipped a signature check unconditionally. Braces would have prevented it. The MISRA-C 15.6 rule mandates braces.

9.2 No unlimited recursion

Recursion eats stack. The stack is small (often 1–8 KB) and shared with ISRs. A recursion depth that depends on input data can silently overflow. Either rewrite iteratively, or impose a hard cap and *verify* that the cap times the per-frame size fits in your stack budget.

9.3 No malloc/free

See previous section. Use one of:

- **Static allocation.** All buffers and objects declared at file scope.

- **Memory pools.** A fixed array of fixed-size blocks plus a bitmask of which are in use. $O(1)$ alloc/free, no fragmentation.
- **Allocator-bounded design.** Allocate everything during `init()`, never afterwards. Then the system either boots or fails to boot — never runs out of memory at runtime.

9.4 Prefer indexes to pointers for array traversal

```
1  /* Less safe */
2  for (int *p = arr; p < arr + N; ++p) *p = 0;
3
4  /* Safer, easier to bounds-check, easier to reason about */
5  for (size_t i = 0; i < N; ++i) arr[i] = 0;
```

Pointers do not carry array length: once you have a `int *p`, all the size information has evaporated. Indexes keep the array name visible at every use, which makes bounds checks both possible and obvious (`if (i < N)`). Tools like static analysers can also reason much better about indices.

9.5 Other classics worth knowing

- No implicit `int` conversions / no shadowed variables.
- No `goto` except for “goto cleanup” patterns.
- No variable-length arrays (`int x[n]` where `n` is runtime — they’re stack-based and unbounded).
- No mixing of signed and unsigned in comparisons.
- Initialise every variable before use; don’t trust “zero by default”.

10 Dedicated Memory Areas for I/O Buffers and DMA Access

DMA (Direct Memory Access) is a hardware engine that copies data between memory and a peripheral (or memory and memory) *without involving the CPU*. The CPU configures source, destination, size, and goes back to doing other things; the DMA controller does the transfer and raises an interrupt when done.

DMA is great for performance (megabytes/s) and power (CPU can sleep), but it introduces constraints that ordinary code does not have.

10.1 Why DMA needs special memory placement

- **Bus accessibility.** The DMA controller has a specific matrix of which masters can reach which memory regions. On STM32F4, for instance, DMA2 can access SRAM1 but not CCM (Core-Coupled Memory). Putting a DMA buffer in CCM yields silent garbage.
- **Cache coherency.** On MCUs with a data cache (Cortex-M7), the CPU sees the cached copy and DMA sees the underlying RAM. If you write through the cache and the DMA reads RAM, it sees stale data; if the DMA writes RAM and the CPU reads the cache, it sees stale data. Solutions: place DMA buffers in a non-cacheable region, or invalidate/clean cache lines manually.
- **Alignment.** Many DMA engines require source/destination alignment to the transfer width (`__attribute__((aligned(4)))`).

- **Linker placement.** You usually carve out a specific region in the linker script (e.g. SRAM2) and force buffers into it via `__attribute__((section(".dma_buffer")))`.

10.2 An example layout

```
1  /* aligned to 32 bytes (cache line), in non-cacheable SRAM2 */
2  __attribute__((aligned(32), section(".dma_buffer")))
3  static uint8_t uart_dma_tx[256];
4
5  __attribute__((aligned(32), section(".dma_buffer")))
6  static uint8_t uart_dma_rx[256];
```

11 The const and volatile Type Modifiers

These two modifiers terrify beginners and yet are absolutely fundamental in embedded. They are independent: a variable can be `const`, `volatile`, both, or neither.

11.1 `const`: “the program promises not to write through this”

- Tells the compiler the value will not change *through this name*. Allows the compiler to put it in read-only memory (Flash) and to propagate the value into uses.
- For pointers, position matters:
 - `const int *p`: pointer to const int. `*p` is read-only, `p` can be reassigned.
 - `int * const p`: const pointer to int. `p` cannot be reassigned, `*p` is mutable.
 - `const int * const p`: both.
- Read it right-to-left: “`p` is a const pointer to a const int”.
- `const` lookup tables in `.rodata` save precious RAM.

11.2 `volatile`: “the value may change without the compiler knowing”

The compiler is allowed to assume that if you don’t write to a variable, its value doesn’t change between reads. `volatile` explicitly revokes that assumption. The compiler must emit an actual load from memory every time you read it, and an actual store every time you write it. It can’t cache it in a register, can’t elide “redundant” reads, can’t reorder across it freely.

When you need it:

1. **Memory-mapped hardware registers.** The peripheral changes them behind your back.
2. **Variables shared with an ISR.** The ISR may write between two reads in main code.
3. **Variables shared between cores / DMA / another bus master.**
4. **Variables modified by `setjmp/longjmp` or signal handlers.**

11.3 `const volatile` — yes, really

A read-only status register: you can read it (compiler must do real load every time), but you must not write it. Example: a hardware “revision ID” register.

```
1 #define CHIP_REV (*(const volatile uint32_t *)0xFFFF7A10)
```

11.4 Famous volatile mistakes

```
1 /* WRONG -- compiler may optimise this to an infinite check that never sees the
   flag change */
2 int flag = 0;
3 void ISR(void) { flag = 1; }
4 void wait(void) { while (!flag); }
5
6 /* RIGHT */
7 volatile int flag = 0;
```

Watch Out

`volatile` does *not* make accesses atomic. Reading a 32-bit `volatile` on a 32-bit MCU is one instruction (atomic), but a 64-bit `volatile` is two stores. Sharing data between an ISR and main code wider than the bus width still requires a critical section.

12 Mandatory Content of `.c` and `.h` Files

C’s compilation model is file-by-file. Each `.c` (*translation unit*) is compiled independently and must therefore see declarations of everything it references from elsewhere. `.h` files are how you share those declarations.

12.1 Header (`.h`) file content

- **Include guard** (or `#pragma once`).
- `#includes` for types this header uses (and *only* those — minimise transitive bloat).
- `typedefs`, `enums`, and `struct` declarations that callers need to know about.
- `extern` declarations for globals exposed by the module.
- Function **prototypes** for the module’s public API.
- Macros and inline functions that are part of the API.
- *No definitions*: no function bodies (except `static inline`), no variable storage. A definition in a header pulled into multiple `.c` files yields multiple-definition link errors.

```
1 #ifndef SENSOR_TEMP_H
2 #define SENSOR_TEMP_H
3
4 #include <stdint.h>
5 #include <stdbool.h>
6
7 typedef struct {
```

```
8     float celsius;
9     uint32_t timestamp_ms;
10 } temp_sample_t;
11
12 bool    temp_init(void);
13 bool    temp_read(temp_sample_t *out);
14 float   temp_to_kelvin(float c);
15
16 #endif /* SENSOR_TEMP_H */
```

Listing 6: A clean header skeleton

12.2 Source (.c) file content

- `#include "own_header.h"` (first, to catch missing includes).
- `#include` other headers as needed.
- **Static** (file-local) helper functions and variables.
- Definitions of the public functions declared in the header.
- No `extern` unless absolutely necessary — if you need something external, declare it in a header.

12.3 Function prototyping

A **prototype** declares a function’s name, return type, and parameter types without a body:

```
1 int compute(int x, int y);    /* prototype */
```

Why it is mandatory in good embedded code:

- Without a prototype, the compiler assumes `int` return type and applies the old K&R promotion rules to arguments. Pass a `float` to an unprototyped function and it’s silently broken.
- It catches type mismatches at the call site.
- Modern C demands prototypes (`-Wmissing-prototypes`, `-Wstrict-prototypes`) and treats `f()` differently from `f(void)` in C (the former still means “unspecified args”).

Note

Rule of thumb: *every* non-**static** function in a `.c` file has a matching prototype in a `.h` file. Every **static** function has a prototype near the top of its `.c` file so functions can call each other freely without forward-declaration headaches.

13 Peripheral Representation in the Memory Map

In a microcontroller, peripherals (UART, SPI, GPIO, TIM, ADC, ...) are not accessed via I/O instructions like on x86. They are **memory-mapped**: each peripheral has a block of addresses that, when read or written by the CPU, actually interact with the hardware.

13.1 Three flavours of register

Almost every peripheral exposes three categories:

Data register (DR).

Where the payload lives. Reading the UART DR yields the received byte. Writing it queues a byte to transmit.

Status register (SR).

Read-only flags reporting what the peripheral is doing: “TX buffer empty”, “RX byte available”, “overrun error”, “transmission complete”. You consult these to decide when to act.

Control register (CR).

Read/write bits that configure the peripheral: enable, baud rate, parity, interrupt enables, DMA enables, polarity, etc.

Larger peripherals add more (BRR for baud, GTPR for guard time, etc.) but the DR/SR/CR pattern is the spine.

13.2 Accessing registers from C: the typedef-struct trick

Vendor headers (e.g. `stm32f4xx.h`) define each peripheral as a `typedef`’d struct whose field offsets match the hardware layout. A `#define` maps a pointer at the peripheral’s base address.

```

1 typedef struct {
2     __IO uint32_t SR;      /* 0x00: Status register      */
3     __IO uint32_t DR;      /* 0x04: Data register        */
4     __IO uint32_t BRR;     /* 0x08: Baud rate register   */
5     __IO uint32_t CR1;     /* 0x0C: Control register 1   */
6     __IO uint32_t CR2;     /* 0x10: Control register 2   */
7     __IO uint32_t CR3;     /* 0x14: Control register 3   */
8     __IO uint32_t GTPR;    /* 0x18: Guard time/prescaler */
9 } USART_TypeDef;
10
11 #define USART2 ((USART_TypeDef *) 0x40004400)
12
13 /* __IO expands to "volatile" -- absolutely critical here */

```

Listing 7: Simplified USART definition

Using it is then natural C:

```

1 while (!(USART2->SR & USART_SR_TXE)) { /* wait */ }
2 USART2->DR = byte;

```

Key Idea

The `__IO/volatile` qualifier is non-negotiable on peripheral registers. Without it, the compiler will optimise away your poll loop or fold multiple writes into one — and the chip will behave incomprehensibly.

13.3 Bit fields you’ll meet

- **r/w bits:** standard read/write.
- **r-only bits:** writes ignored. Status bits often.

- **w-only bits**: reads undefined. Control sequences sometimes.
- **rc_w0 / rc_w1 (read, clear by writing 0/1)**: a flag is set by hardware; you clear it by writing the specified value back. Very common for interrupt flags.
- **rs (set by reading)**: reading a register can change state (e.g. reading UART DR clears RXNE). This is why “just looking at it” in the debugger can change behaviour.

14 STM32CubeIDE

STM32CubeIDE is ST Microelectronics’ official Eclipse-based IDE for their STM32 family. It is the most common environment in coursework because it bundles everything you need and is free.

14.1 What’s inside

- **Editor / build system**. Standard Eclipse CDT with managed Makefile generation.
- **Toolchain**. The `arm-none-eabi-` GCC family, pre-configured.
- **CubeMX (integrated)**. A graphical configurator: you pick your chip, drag pins to peripherals, set clock trees, enable middleware (FreeRTOS, LwIP, FatFS, USB), and it *generates* the initialisation C code, the linker script, and the startup file.
- **Debugger**. GDB front-end, talks to ST-Link / J-Link probes, supports breakpoints, watches, peripheral register view, RTOS-aware task list.
- **HAL/LL libraries**. Shipped as source so you can read and modify them.

14.2 Typical workflow

1. **New project** → pick board or MCU.
2. In the *Pinout & Configuration* view, enable peripherals (e.g. “USART2 in Asynchronous mode”, “TIM3 in PWM Generation on CH1”).
3. In *Clock Configuration*, set your core, AHB, APB1, APB2 clocks. Mismatches here are the source of many “my UART speaks gibberish” bugs.
4. Click *Generate Code*. The IDE produces `main.c` (with markers like `USER CODE BEGIN/END` you must write inside, otherwise re-generation overwrites your edits), `stm32xxx_hal_msp.c`, the startup, and the linker script.
5. Add your application code inside the user markers.
6. Build. Connect probe. *Debug* button flashes and starts a debug session.

14.3 FreeRTOS integration

Inside CubeMX, enable *Middleware* → *FREERTOS*. You get:

- Pre-configured `FreeRTOSConfig.h`.
- CMSIS-OS wrapper around FreeRTOS for portability.

- GUI to declare tasks, queues, mutexes, timers; CubeMX emits the `osThreadNew()` etc. calls.
- Automatic re-assignment of SysTick to FreeRTOS (and a different timer to HAL's tick source — a classic pitfall).

15 HAL vs. LL Libraries; Callbacks; Weak Symbols

ST ships two parallel driver libraries for STM32 peripherals. Knowing which is which and when to use which is half the battle.

15.1 HAL (Hardware Abstraction Layer)

- High-level, “opinionated” API: `HAL_UART_Transmit(&huart2, buf, len, timeout)`.
- Hides peripheral state behind “handles” (`UART_HandleTypeDef`).
- Same API across the whole STM32 family — code portability between chips.
- Provides blocking, interrupt, and DMA variants of every operation.
- **Drawbacks:** larger code size, more cycles per call, sometimes opaque (you can't easily tell which registers it touches), the state machines occasionally have surprises (e.g. “`HAL_BUSY`” returned when you didn't expect).

15.2 LL (Low-Level)

- Thin, inline wrappers over register accesses: `LL_USART_TransmitData8(USART2, byte)`;
- No state machines; almost no overhead — often compiles to the same instructions as direct register code.
- Header-only, can be mixed with HAL on the same project.
- **Drawbacks:** you handle state yourself; less portability across families; less “one-liner” style operations.

Note

Common pragmatic choice: HAL for *configuration* (cleaner, runs once at boot), LL or raw registers for *hot paths* (ISRs, tight loops).

15.3 Interrupt handling through HAL/LL

With HAL. You enable the IRQ in CubeMX. The IRQ handler in `stm32xxxx_it.c` calls e.g. `HAL_UART_IRQHandler(&huart2)`, which detects which event fired, processes it, and then calls a *callback* that you override:

```
1  /* In your code: */
2  void HAL_UART_RxCpltCallback(UART_HandleTypeDef *h)
3  {
4      if (h->Instance == USART2) {
5          rx_byte_ready = 1;
6      }
7  }
```

With LL. You write the IRQ handler directly:

```

1 void USART2_IRQHandler(void)
2 {
3     if (LL_USART_IsActiveFlag_RXNE(USART2)) {
4         rx_buf[rx_head++] = LL_USART_ReceiveData8(USART2);
5     }
6 }

```

15.4 Callbacks: the concept

A **callback** is a function pointer that some library code will call when an event occurs. The library defines *when*, you define *what*. HAL uses callbacks pervasively for ISR events — it provides the dispatch, you write the handler.

15.5 Weak symbols

ST declares the default callback as “weak”:

```

1 __attribute__((weak)) void HAL_UART_RxCpltCallback(UART_HandleTypeDef *h)
2 {
3     UNUSED(h);
4     /* default: do nothing */
5 }

```

A **weak symbol** is overridable: if any other translation unit defines the same symbol *without weak*, the linker uses the strong one and silently ignores the weak one. If nobody defines it, the weak default is used. This is how default ISR handlers and default callbacks work:

- Vendor provides a weak `Default_Handler` as the address of every ISR vector slot. Spurious interrupts thus land in a sensible (if boring) infinite loop instead of executing garbage.
- You define a strong `TIM3_IRQHandler` and the linker transparently replaces the weak default.

Key Idea

Weak symbols let a library ship sensible defaults while letting any user project override them without modifying library files. They are the C equivalent of “virtual methods with a default implementation”.

16 Sensor Interfacing: NTC Thermistor Example

A sensor produces a physical quantity (temperature, light, pressure) that must be converted to a digital number. For a passive **analog sensor** like an NTC thermistor, the path is:

Temperature → Resistance → Voltage → ADC code → Temperature (in software)

16.1 NTC: what it is

An NTC (*Negative Temperature Coefficient*) thermistor is a resistor whose resistance decreases as temperature increases. A 10 kΩ NTC has $R = 10\text{ k}\Omega$ at 25 °C, perhaps 32 kΩ at 0 °C and 4 kΩ at

60 °C. The relationship is strongly nonlinear, usually approximated by the **Beta equation**:

$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{\beta} \ln\left(\frac{R}{R_0}\right)$$

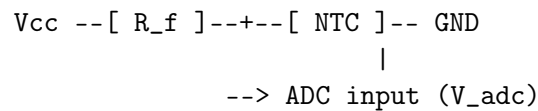
where T is in Kelvin, $T_0 = 298.15$ K, R_0 is the resistance at T_0 , and β (Beta) is a material constant in the datasheet (typically 3000–4500 K).

A more accurate (and more compute-hungry) model is the *Steinhart–Hart* equation:

$$\frac{1}{T} = A + B \ln(R) + C(\ln R)^3.$$

16.2 Resistance-to-voltage: the voltage divider

The ADC reads voltage, not resistance. The simplest converter is a **voltage divider**: NTC in series with a known reference resistor R_f , the junction feeding the ADC.



By the divider rule:

$$V_{\text{adc}} = V_{cc} \cdot \frac{R_{NTC}}{R_f + R_{NTC}} \quad \Rightarrow \quad R_{NTC} = R_f \cdot \frac{V_{\text{adc}}}{V_{cc} - V_{\text{adc}}}.$$

The ADC then produces a code $D = \frac{V_{\text{adc}}}{V_{\text{ref}}} \cdot (2^N - 1)$ where N is the resolution. If $V_{\text{ref}} = V_{cc}$ they cancel out:

$$R_{NTC} = R_f \cdot \frac{D}{(2^N - 1) - D}.$$

16.3 Design choices

- Choose R_f near R_0 to centre the divider's sensitive region around your temperature of interest.
- Watch self-heating: current through the NTC heats it. Keep the bias current small (< 0.1 mA usually).
- ADC noise: take multiple samples and average; or use the MCU's oversampling feature.
- Decoupling: a small capacitor (e.g. 10 nF) across the NTC smooths the input to the ADC sample-and-hold.

17 Fixed-Point vs. Floating-Point

How you represent fractional numbers has *huge* performance and accuracy implications on small MCUs.

17.1 Floating-point: IEEE 754

A float is $\text{sign} \cdot \text{mantissa} \cdot 2^{\text{exponent}}$, encoded in 32 bits (`float`) or 64 bits (`double`).

- Huge dynamic range ($\approx 10^{-38}$ to 10^{+38} for float).
- Relative precision: 7 decimal digits for float, 15 for double.
- Arithmetic is non-associative ($(a+b)+c \neq a+(b+c)$ in general).

Hardware support (FPU): Cortex-M4F/M7/M33 have a single-precision FPU. `float` ops are one or a few cycles. Double precision is software-emulated on M4F (slow), one-cycle on M7 with double-precision FPU.

Without an FPU (Cortex-M0/M0+/M3, AVR), all float arithmetic is implemented in software — a single multiplication costs hundreds of cycles and *kilobytes* of code (`libgcc` routines).

17.2 Fixed-point: integers with an implied scale

Pick a scale factor (power of two for efficiency) and interpret integers as multiples of it. **Q-format** names them: `Q16.16` means 32 bits, 16 integer bits, 16 fractional bits, so the LSB represents 2^{-16} .

- Addition/subtraction: plain integer add/sub.
- Multiplication of `Qm.n` by `Qm.n` gives `Q(2m).(2n)` — you must shift right by n to renormalise (and use 64-bit intermediate to avoid overflow).
- Division: shift left by n first, then integer divide.

17.3 When to use which

- **Cortex-M0/M3, AVR, no FPU**: fixed-point almost always.
- **Cortex-M4F+**: single-precision float is fine for most things and easier to reason about. Stick to `float`, not `double`, to keep using the FPU.
- **DSP / filtering at high rate**: fixed-point (often `Q15` or `Q31`) lets you use the SIMD/DSP instructions (Cortex-M4/M7 DSP extension, ARM CMSIS-DSP library).
- **Algorithms with wide dynamic range**: float, unless you really know your numeric range.

Watch Out

Beware of accidental promotions. `x / 2` with integer `x` is integer division; `x / 2.0` promotes to double, drags in the double-precision soft-float library, and bloats your binary. Write `x / 2.0f` or just `x >> 1`.

18 FreeRTOS Best Practices: Tasks, Queues, Events

FreeRTOS is a small preemptive RTOS. It gives you tasks (with priorities), inter-task communication primitives, and synchronisation primitives. The kernel is tiny (a few kB) and well understood.

18.1 Tasks

A **task** is a function with its own stack and priority that runs as if it owns the CPU. The scheduler picks the highest-priority *ready* task and runs it; if a task blocks (waits on a queue, sleeps, waits on a semaphore), the next-highest ready task runs.

Best practices:

- **Each task does one thing well.** “UART RX task”, “Motor control task”, “Display update task”. Resist the temptation to make one giant task with a state machine.
- **Tasks block, they don’t poll.** A loop with `vTaskDelay` is still polling. Block on the actual event (queue receive, semaphore take). The scheduler can then run other work or enter low-power.
- **Assign priorities by deadline criticality, not importance.** A 1-ms control loop is higher priority than the LCD redraw, even if the LCD “feels” more important.
- **Size stacks deliberately.** Use `uxTaskGetStackHighWaterMark` to measure actual usage. Allocate a bit of headroom but not megabytes.
- **Avoid `vTaskDelay` for timing of work.** Use `vTaskDelayUntil` for periodic tasks — it compensates for the execution time of the loop body, giving you actual period accuracy.

18.2 Queues

A **queue** is a thread-safe FIFO. Producers `xQueueSend`, consumers `xQueueReceive`; both can block with timeouts.

- Pass copies of small structures through queues, not pointers to stack-local data.
- Use queues to convert ISR events into task work: `xQueueSendFromISR` in the ISR, `xQueueReceive` in the consumer task. The ISR stays short; the heavy work happens in task context.
- Use `portYIELD_FROM_ISR(xHigherPriorityTaskWoken)` at the end of the ISR so the consumer task can run immediately if it has higher priority than the interrupted task.

18.3 Events and other primitives

Binary semaphore.

One-bit signal. “Something happened, go do it.” Released from ISR, taken by a task.

Counting semaphore.

Counts resources or events.

Mutex.

Mutual exclusion, with *priority inheritance* (a low-prio task holding the mutex temporarily inherits the priority of a high-prio waiter, to avoid priority inversion).

Event group.

A 24-bit set of flags; a task can wait for “any of these bits” or “all of these bits”. Good for “wait for either serial-done or timeout”.

Direct task notifications.

The cheapest of all — a per-task 32-bit value with built-in semaphore semantics. Use them when only one task is waiting; they avoid allocating a queue/semaphore object.

Watch Out

Two notorious classes of RTOS bugs: **priority inversion** (use mutexes, not binary semaphores, for resource protection) and **stack overflow** (turn on `configCHECK_FOR_STACK_OVERFLOW = 2` during development).

19 Timer Peripheral: Prescaler and Period

A hardware timer is, at its core, a counter clocked from the system clock (possibly divided). It counts from 0 up to a programmable maximum, then either wraps or stops, and emits events (interrupt, DMA request, output toggle) at programmable points.

19.1 The two knobs

Prescaler (PSC).

An integer divider applied to the timer's input clock before it reaches the counter. If the timer clock is f_{clk} and the prescaler is PSC, the counter ticks at

$$f_{\text{count}} = \frac{f_{\text{clk}}}{\text{PSC} + 1}.$$

(The +1 is because the register value 0 means “divide by 1”.)

Period / Auto-Reload Register (ARR).

The value at which the counter wraps (or the event fires). The update event happens every $\text{ARR} + 1$ counter ticks, so the update frequency is

$$f_{\text{update}} = \frac{f_{\text{clk}}}{(\text{PSC} + 1)(\text{ARR} + 1)}.$$

19.2 Worked example

Goal: produce a 1 kHz tick from a 72 MHz timer clock.

$$\frac{72\,000\,000}{1\,000} = 72\,000 \text{ counter ticks per period.}$$

A single 16-bit ARR maxes at 65535, so 72 000 won't fit. Insert a small prescaler:

- $\text{PSC} = 71 \Rightarrow$ counter ticks at $72\text{MHz}/72 = 1 \text{ MHz}$.
- $\text{ARR} = 999 \Rightarrow$ update event every 1000 ticks $\Rightarrow 1 \text{ kHz}$. Done.

19.3 What you actually do with timers

- **Periodic interrupt** (“tick”). Set PSC and ARR, enable update interrupt, run a handler at fixed cadence.
- **PWM output** (Output Compare mode). Counter compares against another register (CCR). Output flips at compare match, resets at ARR wrap $\Rightarrow$ duty cycle = $\text{CCR}/(\text{ARR}+1)$.
- **Input capture**. Latch the counter value on an edge of an input pin — measure pulse widths, frequencies, etc.

- **Encoder mode.** Two inputs decoded as quadrature: counter automatically tracks position of a rotary encoder.
- **Triggering ADC / DMA.** Timers can request a DMA transfer or start an ADC conversion at exact moments without CPU involvement. Foundation of high-rate, jitter-free signal acquisition.

20 Power Electronics from Scratch

Before the switching sections, the handful of device ideas they assume — built from the bottom up. Nothing new, just the floor.

A transistor is a controlled valve. A GPIO sources a few mA at 3.3 V; motors/heaters/lamps want amps at 12–48 V. You don’t drive the load from the pin — you drive a transistor that drives the load. The *control terminal* (gate/base) is the handle the pin works for almost nothing; the *main path* (drain–source / collector–emitter) is the pipe that carries the load current.

Two kinds of valve. A *MOSFET* is *voltage*-operated: gate voltage (vs. its source) above V_{th} opens the channel, and holding it open costs $\approx$ no current (a “logic-level” FET opens at 3.3 V). A *BJT* is *current*-operated: $I_C = \beta I_B$ in the active region ($\beta \approx 50$ –200), and it needs continuous base current to stay on. That difference is why MOSFETs dominate switching and why a BJT’s base drive is a real design step (below).

What turns a MOSFET on: gate-to-source. The channel opens when $V_{GS} = V_G - V_S \geq V_{th}$ — never gate-to-ground. So the whole question is *where the source sits*. Source at ground (low-side): a 3.3 V pin is 3.3 V of V_{GS} , trivial. Source up at V_{cc} (high-side): the gate must go *above* V_{cc} (a 12 V rail wants ~ 17 V), which a logic pin can’t make alone.

Note

“It’s gate-to-source, so where’s the source?” is the seed of the entire low-/high-side story — and the reason smart high-side drivers exist.

BJT switch = overdriven into saturation. An amplifier stays in the active region; a *switch* is driven hard until V_{CE} bottoms out at $V_{CE(sat)} \approx 0.1$ –0.3 V (saturation — a closed switch with a small drop). For margin you *overdrive*: design to a low “forced β ” of 10–20 (feed $\sim I_C/15$) instead of the real $\beta \approx 100$. That “forced β ” is what sizes the base resistor (below).

Making a voltage above the rail. A high-side N-channel gate needs a voltage above V_{cc} that isn’t on the board — so a *charge pump* / *bootstrap* capacitor makes one. You rarely build it; it lives inside integrated high-side switches.

Why switching wastes energy. Fully on, $V \approx 0$; fully off, $I \approx 0$ — so $P = VI \approx 0$ in both steady states. Loss happens only *during the flip*, when V and I are briefly both present, giving the $P_{sw} \approx \frac{1}{2}VI(t_r + t_f)f_{sw}$ you’ll meet in the next section. Faster edges = less loss but more EMI.

Note

Heat is Ohm’s law renamed: $\Delta T = P \cdot R_{th}$ (power $\leftrightarrow$ current, $R_{th} \leftrightarrow$ resistance, temperature rise $\leftrightarrow$ voltage), so $T_j = T_{amb} + P R_{th}$. A heatsink is just a smaller R_{th} .

21 Load Switches: Low-Side and High-Side

A **load switch** is a transistor used to turn a load (motor, LED, heater, relay coil) on and off. Where in the circuit you place the switch fundamentally changes the design.

21.1 Low-side switch

The switch sits *between the load and ground*. The load is permanently connected to Vcc.

Vcc --[LOAD]--+-+-- DRAIN/COLLECTOR SWITCH SOURCE/EMITTER --GND

To turn on:

- **NMOS as low-side:** drive the gate above the source (which is at ground), so $V_{GS} \approx V_{cc-\text{logic}}$. A 3.3 V logic gate is plenty for a “logic-level” NMOS (e.g. AO3400, IRLML2502).
- **NPN BJT as low-side:** drive base current into the base resistor; $V_{BE} \approx 0.7 \text{ V}$; saturate the transistor.

Advantages. The control signal is referenced to ground, same as the MCU. Easy to drive directly from a GPIO. Simple gate/base drive.

Disadvantages. The load is “hot” even when off (Vcc always present at one terminal). If the load’s downstream terminal accidentally shorts to ground, the switch can’t protect against it. Common-ground EMI considerations.

21.2 High-side switch

The switch sits *between Vcc and the load*. The load is permanently connected to ground.

Vcc -- SOURCE/EMITTER SWITCH DRAIN/COLLECTOR --+-- [LOAD] -- GND

To turn on:

- **PMOS as high-side:** gate must be *below* source by $|V_{GS(th)}|$. The source is at Vcc, so you must pull the gate down — typically to GND. Driving a PMOS from a 3.3 V MCU with Vcc=12 V requires a level-shifter (an NPN/NMOS pulling the gate low).
- **NMOS as high-side:** source sits at the load, near Vcc when on. To keep it on you need $V_{GS} > V_{th}$, i.e. gate at $V_{cc} + V_{GS}$. That voltage doesn’t exist on the board — you must create it with a *charge pump* or a *bootstrap capacitor*.

Advantages. The load is fully isolated from Vcc when off — safer. A short-to-ground at the load side simply pulls more current through the switch, which can be sensed and limited. Many industrial applications mandate high-side switching for this reason.

Disadvantages. Drive circuitry is more complex. This is why integrated high-side drivers exist (next section).

21.3 NPN BJT as low-side switch: detailed walk-through

An NPN turns on when $V_{BE} \gtrsim 0.6 \text{ V}$ and sufficient base current flows. The relationship is:

$$I_C = \beta \cdot I_B,$$

but only in the active region. To use a BJT as a switch we want it *saturated* (very low V_{CE} , $\sim 0.2 \text{ V}$), which requires **overdriving** the base, typically $I_B \approx I_C/10$ (a “forced beta” of 10, well below the device’s β of perhaps 100).

22 Integrated High-Side Drivers

Because high-side NMOS driving is non-trivial, vendors sell single-chip **high-side drivers**. Examples: BTS50xx (Infineon Profet), TPS272xx (TI), VN5T family. Inside one tiny package you get:

- A power NMOS (or a clever PMOS stack) for the switch itself.
- A **charge pump** that generates a gate voltage above V_{cc} so the NMOS can be fully enhanced.
- **Current sense** (a small “sense FET” mirror or a low-side shunt) producing a proportional analog output (IS pin) for the MCU’s ADC.
- **Current limit / over-current shutdown**, often configurable via an external resistor.
- **Thermal shutdown** when the die overheats.
- **Reverse-battery protection** in automotive variants.
- **Open-load detection** (in OFF state) for diagnostics.
- **Slew rate control**: the driver shapes the gate drive so that turn-on/turn-off is neither too slow (high $P = V \cdot I$ during transition $\Rightarrow$ heat) nor too fast (EMI, ringing on inductive loads).

22.1 Why turn-on/turn-off speed matters

Too slow. The transistor spends time in its active region with both high V_{DS} and high I_D — power dissipation $P = V \cdot I$ peaks. A 1 A, 12 V load with a 1 μ s transition wastes $\sim 6 \mu$ J per edge. At 100 kHz PWM that’s 1.2 W just from switching losses.

Too fast. Sharp dI/dt across parasitic inductance generates voltage spikes that ring, radiate (EMI), and can damage the FET or trigger spurious turn-ons of the opposite-side switch in a bridge.

Slew-rate-controlled drivers compromise by ramping the gate charging current, achieving a controlled (often programmable) dV/dt .

23 Switch Power Dissipation; NPN Base Resistor Sizing

23.1 Where the power goes (MOSFET)

- **Conduction loss:** $P_{\text{cond}} = I_D^2 \cdot R_{DS(on)}$. Buy a FET with low $R_{DS(on)}$ at the gate voltage you can supply.
- **Switching loss:** energy dissipated during transitions. Roughly $P_{\text{sw}} = \frac{1}{2} V_{DS} \cdot I_D \cdot (t_r + t_f) \cdot f_{\text{sw}}$.
- **Gate-drive loss:** $P_{\text{gate}} = Q_g \cdot V_{GS} \cdot f_{\text{sw}}$. Modest at low frequencies, can matter at MHz.
- **Body-diode loss:** in synchronous topologies, current may briefly flow through the body diode, which has a higher V_F than the channel.

23.2 Where the power goes (BJT)

- **Conduction loss:** $P_{\text{cond}} = V_{CE(\text{sat})} \cdot I_C$. $V_{CE(\text{sat})}$ is typically 0.1–0.3 V. Unlike a MOSFET this is a voltage drop, not an ohmic resistance: doubling the current does not quadruple the loss, only doubles it.
- **Base drive loss:** $P_{\text{base}} = V_{BE} \cdot I_B$ (small but non-zero).

23.3 Sizing an NPN base resistor

Given:

- GPIO logic level: V_{IO} (e.g. 3.3 V).
- BJT V_{BE} when saturated: ≈ 0.7 V.
- Desired collector current: I_C .
- Worst-case β_{min} from the datasheet (use this, not typical!).
- Overdrive factor (forced beta): pick $\beta_{\text{forced}} = 10$ or so, to guarantee saturation across temperature/lot variation.

Required base current:

$$I_B = \frac{I_C}{\beta_{\text{forced}}}.$$

Voltage available across the resistor:

$$V_R = V_{\text{IO}} - V_{BE}.$$

Resistor value:

$$R_B = \frac{V_R}{I_B} = \frac{V_{\text{IO}} - V_{BE}}{I_C / \beta_{\text{forced}}}.$$

Numerical example. 3.3 V GPIO, load 100 mA, choose $\beta_{\text{forced}} = 10$:

$$I_B = 10 \text{ mA}, \quad V_R = 2.6 \text{ V}, \quad R_B = 260 \Omega.$$

Use 220Ω (next standard E12 value down — slight over-drive is fine). Check the GPIO can source 10 mA (most can, but at the edge of spec).

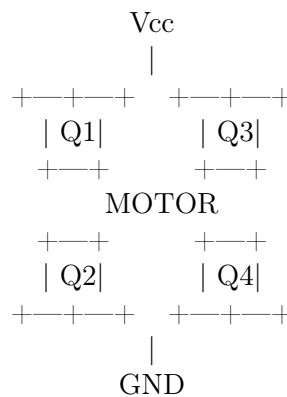
Watch Out

Never drive a BJT base directly from a GPIO without a series resistor — you'll exceed the device's max base current and either kill the BJT or kill the GPIO output stage.

24 Full-Bridges and Shoot-Through

A **full-bridge (H-bridge)** is four switches arranged so a DC voltage can be applied to a load in either polarity. The classical use: driving a DC motor forwards and backwards.

24.1 Topology



Q1 (high-side left) and Q4 (low-side right) on $\Rightarrow$ current flows left-to-right through the load. Q3 + Q2 on $\Rightarrow$ current right-to-left. PWM Q1 vs. Q2 (or Q3 vs. Q4) and you have direction + speed control.

24.2 Shoot-through: the cardinal sin

Shoot-through occurs when Q1 *and* Q2 are both on simultaneously (or Q3 and Q4), creating a direct Vcc-to-GND path through two transistors. Currents of hundreds of amps can flow for microseconds. The result: instant catastrophic failure — molten silicon, flames, all of the above.

This is rarely a steady-state problem (control logic won't intentionally command it). It happens during *transitions*: when you ask Q1 to turn off and Q2 to turn on, the real-world devices don't switch instantly. If Q1's turn-off is slower than Q2's turn-on, there's an overlap window where both are partially on.

24.3 Protection: dead-time

The solution is **dead-time**: insert a short period where *both* switches in a leg are off, between turning one off and turning the other on. During dead-time, current continues to flow through the body diodes of the off switches (or through external freewheel diodes). Typical dead-times: 100 ns to a few μ s, depending on device speed.

24.3.1 Timer-based dead-time

Advanced timers in modern MCUs (STM32 TIM1, TIM8) support **complementary outputs with programmable dead-time** natively. You configure one PWM channel and the timer automatically generates a complement on a second pin, separated by your specified dead-time (register DTG — “Dead-Time Generator”). No software fiddling, hardware guarantees the gap.

24.3.2 Driver-based dead-time

Half-bridge gate driver chips (e.g. IR2104, DRV8301, UCC27210) include built-in dead-time insertion. You provide a single PWM signal; the driver generates the high-side and low-side gate signals with an internal interlock that physically prevents both from being asserted simultaneously.

Key Idea

Belt-and-braces is normal: *both* the timer and the driver enforce dead-time. A software bug that disables one is then still caught by the other.

25 Digital-Interface Sensors: The Accelerometer Example

A **digital-interface sensor** performs the analog-to-digital conversion internally and exposes its measurements via a digital bus (I²C, SPI, sometimes UART or a 1-wire variant). The MCU just reads registers.

25.1 Why digital is nicer

- No noisy analog wires snaking across the board.
- Calibration done once at the factory and baked into the sensor.
- Configurable filters, ranges, sample rates accessible via registers.
- Built-in features: temperature compensation, motion-detection interrupts, FIFOs.

25.2 MEMS accelerometer, briefly

An accelerometer measures linear acceleration on 1, 2, or 3 axes. A MEMS device contains a tiny silicon proof mass on springs (etched into the chip); acceleration deflects the mass, changing capacitances measured by internal circuitry. Output: g -force values in the configured range (e.g. $\pm 2g, \pm 4g, \pm 8g, \pm 16g$).

25.3 Example: hard disk drives and free-fall detection

Many laptop hard drives ship with a 3-axis accelerometer. Use case: if the drive senses *free fall* (all three axes near $0g$ simultaneously, i.e. no gravity reaction — only happens in free fall), firmware concludes the drive is falling. Within milliseconds it parks the head off the platter, so when the laptop hits the floor the platter and head do not collide.

The detection is built into the sensor: registers configure a free-fall threshold and duration; an interrupt pin asserts when the condition holds. The MCU's ISR triggers the head-park sequence. The whole loop is fast precisely because the heavy lifting is in the sensor.

Other classic uses: smartphones (rotate the display), pedometers, vibration monitoring of industrial machinery, automotive airbag triggering.

26 I²C and SPI Protocols

These are the two ubiquitous embedded buses for chip-to-chip communication on a board. Memorise them.

26.1 I²C (Inter-Integrated Circuit)

A 2-wire, multi-master, multi-slave bus.

Wires

SDA (Serial Data, bidirectional, open-drain) and SCL (Serial Clock, open-drain). Pulled up to Vcc by external resistors (typically 4.7–10 k Ω).

Speeds

Standard 100 kHz, Fast 400 kHz, Fast+ 1 MHz, High-Speed 3.4 MHz.

Addressing

Each slave has a 7-bit address (some support 10-bit). A transaction starts with the master pulling SDA low while SCL is high (*START condition*), then clocking out the 7-bit address + 1 R/W bit. The matching slave ACKs by pulling SDA low for one clock.

Direction

Set by the R/W bit. Reads typically require: send register address (write phase), *repeated START*, then read.

Stop

Master releases SDA from low to high while SCL is high (*STOP condition*).

Advantages. Only 2 wires for an entire bus of many devices. Standard, supported everywhere. Software-emulatable on plain GPIO.

Drawbacks. Slower than SPI. Open-drain means rise time is limited by the pull-up resistor and the bus capacitance. Bus can hang if a slave holds SDA low (recovery: toggle SCL nine times). Addressing collisions between vendors — watch the address map of every chip you put on the bus.

26.2 SPI (Serial Peripheral Interface)

A 4-wire (sometimes 3-wire), full-duplex, master-slave bus.

Wires

SCK (clock, master → slave), MOSI (Master Out Slave In), MISO (Master In Slave Out), and one CS (Chip Select) per slave (active low). Push-pull, no pull-ups needed.

Speeds

Tens of MHz easily, up to 100+ MHz on capable devices.

Operation

Master asserts CS, then shifts bits out on MOSI on one clock edge while the slave shifts back on MISO on the other edge. Every clock = one bit each direction; full-duplex.

Modes

Four combinations of clock polarity (CPOL) and clock phase (CPHA): Mode 0 to Mode 3. The chosen mode must match between master and slave. Wrong mode = nothing works, often silently.

Daisy-chaining

Some devices can be chained: MISO of one goes to MOSI of the next. Useful for shift registers.

Advantages. Very fast, simple state machine, no addressing overhead, push-pull drives don't care about pull-ups.

Drawbacks. One CS line per slave (more pins). No standard framing or error detection — it's just shift registers, you pay attention to bytes. No flow control — slave must be ready every clock. No defined slave-to-master interrupt mechanism (often a separate GPIO "INT" pin handles this).

26.3 Comparison cheat sheet

	I ² C	SPI
Wires (n slaves)	2 total	3 + n (one CS each)
Speed	up to a few MHz	tens to 100+ MHz
Multi-master	Yes (arbitration)	No (one master typically)
Addressing	7/10-bit on bus	Per-slave CS line
Duplex	Half (one direction at a time)	Full
Framing/ACK	Built-in	None (raw bits)
Drive type	Open-drain + pull-up	Push-pull
Typical use	Sensors, EEPROMs, small ICs	SD card, displays, ADCs, fast sensors

26.4 In an STM32 (peripherals view)

Each MCU has hardware peripherals (e.g. I2C1, I2C2, I2C3, SPI1, SPI2, SPI3) with their own DMA channels and interrupts. Workflow:

1. In CubeMX, enable the peripheral, set pins, baud/speed, SPI mode, optional DMA.
2. In code, use HAL or LL: `HAL_I2C_Mem_Read(&hi2c1, addr<<1, reg, 1, buf, n, timeout)` for I²C reads.
3. For high-rate transfers, use the `_DMA` variants and handle the completion callback to chain the next transaction.

Note

A subtle gotcha: HAL's `HAL_I2C_*` functions expect the slave address *shifted left by 1*, because the LSB is used internally for the R/W bit. Forgetting the shift gives “no ACK” and confused debugging.

Closing Thoughts

This syllabus is broad on purpose: firmware design lives at the intersection of *C language*, *computer architecture*, *electronics*, and *real-time systems*. The recurring themes:

- Embedded code runs against *physics*, not a forgiving OS. Determinism, resource budgets, and correct hardware interaction matter more than elegance.
- Understanding what the compiler and linker actually do (sections, optimisation, calling conventions, weak symbols) is not optional — it's the only way to debug “the C is fine but the chip is wrong”.
- Hardware peripherals are just registers. Once you read the reference manual you can drive anything; libraries are conveniences, not magic.
- Patterns repeat: ISR pushes data to a queue, task processes it; timer triggers ADC via DMA; weak default ISR overridden by your strong one; mutex protects shared state with priority inheritance. Recognising these patterns lets you read any embedded code base.

Now go flash a board and break something. That's how this stuff actually sticks.